Task340. Брокеры сообщений (на примере Apache Kafka)

Дано:

Разрабатываемая система обрабатывает сущности **Author**, **Tweet**, **Marker** и **Reaction**, которые логически связаны отношениями (см. предыдущие задачи)

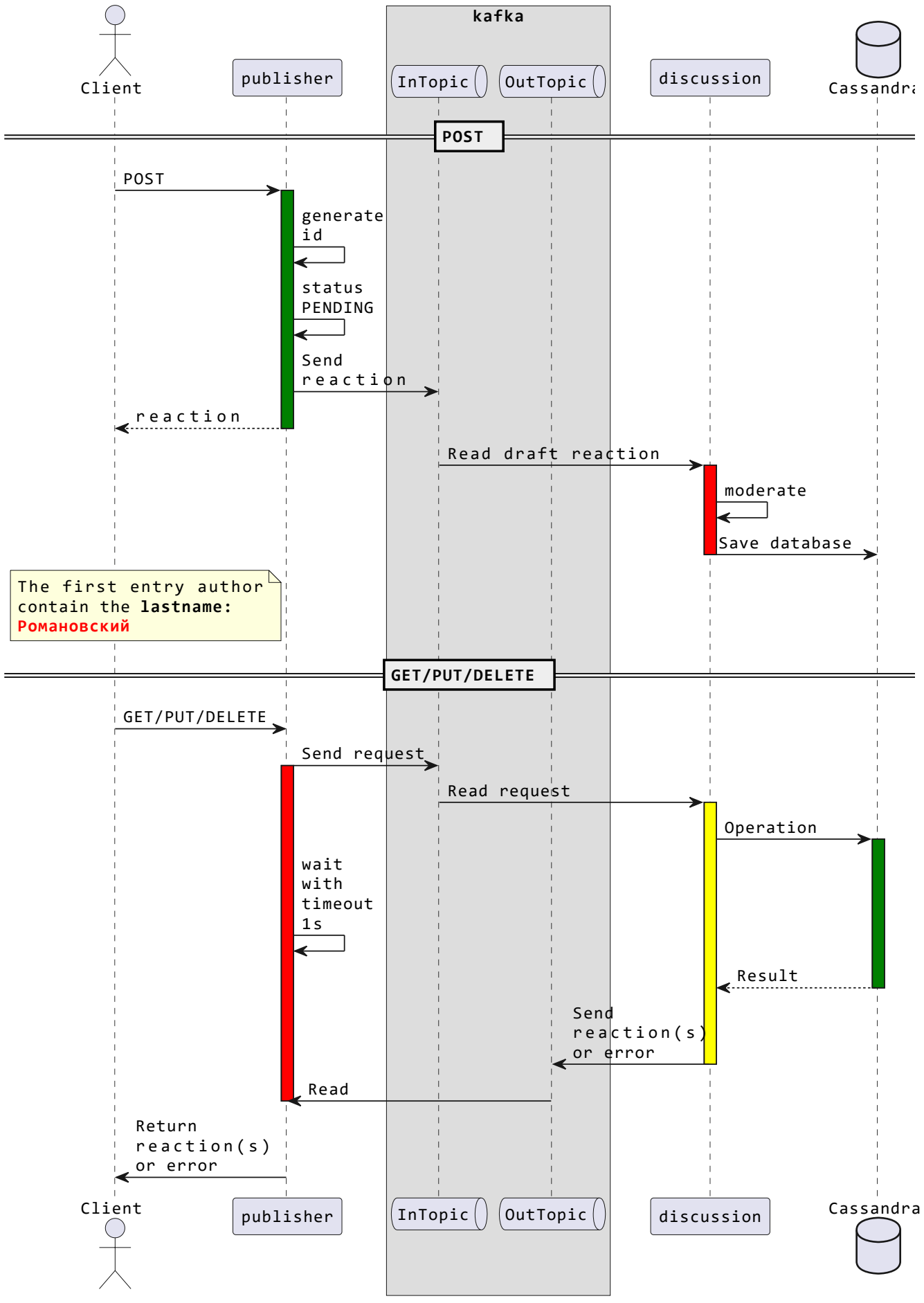
- один-ко-многим (**Author** и **Tweet**, **Tweet** и **Reaction**)
- многие-ко-многим (**Tweet**, **Marker**).

Существует и работает REST-передача между модулями **publisher** и **discussion** сущности **Reaction**

Задание

Необходимо в ранее разработанном распределенном приложении реализовать транспор между модулями **publisher** и **discussion** для сущности **Reaction** через топики Kafka.

Основой для выполнения задания является следующая диаграмма взаимодействия



Технические требования

Используйте префикс **/api/v1.0/** для контроллеров REST и их методов,

Используйте адрес и порт **localhost:24110** для самого приложения (это модуль **publisher**).

Не отключайте REST **localhost:24130** для модуля **discussion**.

Используйте обязательный префикс **tbl_** для таблиц(ы) в базе(ах) данных

Используйте подключение по умолчанию к **Kafka** из hub.docker.com

Пример запуска:

```
docker network create kafkanet

docker run -d --network=kafkanet --name=zookeeper -e \
ZOOKEEPER_CLIENT_PORT=2181 -e ZOOKEEPER_TICK_TIME=2000 -p
2181:2181 \
confluentinc/cp-zookeeper

docker run -d --network=kafkanet --name=kafka -e \
KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181 -e \
KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 -e
\
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1 -p 9092:9092 \
confluentinc/cp-kafka
```

Рекомендации

Подготовительные шаги:

- Изучить документацию по Apache Kafka.
- Установить и настроить Apache Kafka локально, используя docker.
- Создать поле state в сущности **Reaction** с тремя состояниями PENDING APPROVE DELCINE
- Создать топики **InTopic** и **OutTopic** для обмена сообщениями или автоматизировать создание топиков при их отсутствии (Spring это делает автоматически)

Интеграция Kafka с publisher:

- Создать Kafka Producer в модуле **publisher** для отправки сообщений в топик **InTopic**.
- Настроить отправку **Reaction** так, чтобы в топике **InTopic** сообщения принадлежащие одной **Tweet** всегда оказывались в одной **partition** (независимо от числа **partition**).
- Создать Kafka Consumer **publisher** для чтения сообщений из **OutTopic**.

Интеграция Kafka с discussion:

- Создать Kafka Consumer в модуле **discussion** для чтения запросов из топика **InTopic**:
  - модуль должен обновлять статус после автоматической модерации на APPROVE или DELCINE
  - алгоритм модерации может быть любым, например на основе стоп-слов
- Создать Kafka Producer в модуле **discussion** для отправки ответов в топик **OutTopic**.

Тестирование и отладка:

- Протестировать передачу сообщений через Kafka.
- Проверить регрессионно работоспособность системы после интеграции Kafka.

Масштабирование и оптимизация:

- Оптимизировать настройки Kafka для повышения производительности.
- Рассмотреть возможность масштабирования Kafka для обеспечения отказоустойчивости.
- Определить минимальное число брокеров, партиций и RF для обеспечения отказоустойчивости кластера Kafka.

Дополнительные шаги (по желанию)

1. Оптимизировать производительность модуля/микросервиса **publisher** (например, использовать кеширование)
2. Настроить тестирование через развертывание базы данных в **Testcontainers**
3. Настроить мониторинг и логирование (тут масса вариантов)

Language:

Connect with us

Get DC-training

Available for Linux, macOS, and Windows platforms

Get started

Community

Telegram

I see errors

Where can I get Demos?

Resources

Docker Hub

Get desktop versions

Video playlist

Open zoom webinar

More

Last release

About me

My LinkedIn

Copyright © 2023 - 2025, Alexander Khmelov. All rights reserved. Java are registered trademarks Oracle. Other names may be trademarks of their respective owners / 9.4.2
/ v.6.1.28

- Практика
- Official intro
- Official Quick Start
- Kafka в KRaft-режиме
- II. Kafka Fundamentals
- III. Spring Kafka Integration
- IV. Kafka Streams Deep Dive
- V. Kafka Connect
- VI. Security and Monitoring
- VII. Advanced Kafka Patterns
- VIII. Kafka in Cloud Environments
- Links & Docs (en)

Stop application